

MACHINE LEARNING ENGINEERING

FASE 4 | AULA 02 -

TEORIA DAS REDES NEURAIS PROFUNDAS

SUMÁRIO

O QUE VEM POR AÍ?	3
HANDS ON	4
SAIBA MAIS.....	5
O QUE VOCÊ VIU NESTA AULA?	30
REFERÊNCIAS.....	32

EMSE

O QUE VEM POR AÍ?

Depois de compreender as bases das redes neurais por meio do estudo dos perceptrons de camada única, podemos avançar para as redes neurais de múltiplas camadas.

À medida que mergulhamos nas profundezas das redes de múltiplas camadas, é crucial entender os princípios de probabilidade que governam o comportamento e o aprendizado desses sistemas.

Além disso, um aspecto técnico fundamental que possibilita essas capacidades avançadas é a seleção e utilização de funções de ativação apropriadas. Essas funções ajudam a determinar se um neurônio deve ser ativado ou não, influenciando diretamente a complexidade dos padrões que a rede pode aprender.

Finalmente, a otimização e o algoritmo de backpropagation são essenciais para ajustar os pesos das conexões neurais de maneira eficiente, permitindo que a rede aprenda de forma eficaz a partir de grandes volumes de dados.

HANDS ON

Boas-vindas à nossa seção prática, na qual colocaremos em ação tudo o que aprendemos sobre redes neurais de múltiplas camadas e suas aplicações avançadas.

Neste módulo, vamos mergulhar nos fundamentos das redes neurais além das camadas únicas, explorando desde os conceitos básicos de probabilidade até técnicas avançadas como inferência Bayesiana, espaços latentes e mais.

Vamos consolidar o entendimento teórico com exemplos práticos e hands-on em C++, uma linguagem poderosa para implementações de baixo nível e que oferece controle preciso sobre os recursos computacionais, além de ser ideal para os casos de implementação de Redes Neurais.

SAIBA MAIS

Começaremos com uma introdução às redes neurais de múltiplas camadas. Essas redes são essenciais para tarefas que requerem a aprendizagem de padrões complexos e não linearmente separáveis. Assim, a implementação a seguir em C++ demonstra a inicialização de uma rede simples:

```
#include <vector>
#include <iostream>

class NeuralNetwork {
public:
    NeuralNetwork(int inputSize, int hiddenSize, int outputSize) {
        // Inicializa pesos e bias aqui (simplificado para exemplo)
    }

    void forward(std::vector<double>& inputs) {
        // Processamento da camada escondida e saída
    }
};

int main() {
    NeuralNetwork nn(3, 5, 2); // Rede com 3 entradas, 5 neurônios na camada escondida,
    e 2 saídas
    std::vector<double> inputs = {1.0, 0.5, -1.0};
    nn.forward(inputs);
    return 0;
}
```

Código-fonte 1 – Código base para criação de Redes Neurais
Fonte: elaborado pelo autor (2024)

A seguir, exploraremos como os princípios de probabilidade são aplicados nas redes neurais para realizar inferências a partir de dados incertos. Em C++, podemos simular a aplicação de probabilidades na ativação de neurônios:

```
#include <cmath>

double sigmoid(double x) {
    return 1.0 / (1.0 + exp(-x));
}
```

Código-fonte 2 – Aplicação de probabilidades na ativação de neurônios
Fonte: elaborado pelo autor (2024)

A inferência Bayesiana permite que redes neurais ajustem suas previsões baseadas em novas evidências. Um exemplo simplificado seria ajustar os pesos da rede com base na probabilidade posterior:

```
double updateWeight(double prior, double likelihood, double evidence) {
    return (likelihood * prior) / evidence;
}
```

Código-fonte 3 – Ajustando os pesos da rede com base na probabilidade posterior
Fonte: elaborado pelo autor (2024)

As funções de ativação determinam se um neurônio deve disparar ou não. Em C++, a função ReLU é comumente usada por sua eficiência em modelos de aprendizado profundo:

```
double relu(double x) {  
    return std::max(0.0, x);  
}
```

Código-fonte 4 – Função ReLU
Fonte: elaborado pelo autor (2024)

O processo de otimização e backpropagation é fundamental para ajustar os pesos da rede neural. Aqui está uma implementação básica do gradiente descendente em C++:

```
void updateWeights(std::vector<double>& weights, std::vector<double>& gradients,  
double learningRate) {  
    for (size_t i = 0; i < weights.size(); ++i) {  
        weights[i] -= learningRate * gradients[i];  
    }  
}
```

Código-fonte 5 – Implementação básica do gradiente descendente em C++
Fonte: elaborado pelo autor (2024)

Finalmente, a regularização é crucial para evitar o sobreajuste. Implementações simples como L2 podem ser feitas ajustando o método de atualização de pesos:

```
void updateWeightsWithL2(std::vector<double>& weights, std::vector<double>&  
gradients, double learningRate, double lambda) {  
    for (size_t i = 0; i < weights.size(); ++i) {  
        weights[i] -= learningRate * (gradients[i] + lambda * weights[i]);  
    }  
}
```

Código-fonte 6 – Ajustando o método de atualização de pesos
Fonte: elaborado pelo autor (2024)

O código completo da implementação de uma rede neural com todas estas características é o seguinte:

```
#include <iostream>  
#include <vector>  
#include <cmath>  
#include <numeric>  
#include <cstdlib>  
#include <ctime>  
  
// Função de ativação sigmoid  
double sigmoid(double x) {  
    return 1.0 / (1.0 + exp(-x));  
}  
  
// Derivada da função sigmoid  
double sigmoid_derivative(double x) {  
    return x * (1.0 - x);  
}
```

```
// Estrutura para uma rede neural
class NeuralNetwork {
private:
    std::vector<double> inputs;
    std::vector<double> hidden;
    std::vector<double> output;
    std::vector<double> target;
    std::vector<std::vector<double>> weights_input_hidden;
    std::vector<std::vector<double>> weights_hidden_output;
    std::vector<double> bias_hidden;
    std::vector<double> bias_output;
    std::vector<double> hidden_delta;
    std::vector<double> output_delta;
    int inputSize, hiddenSize, outputSize;

public:
    NeuralNetwork(int inputs, int hidden, int outputs) : inputSize(inputs),
        hiddenSize(hidden), outputSize(outputs) {
        // Inicializa as camadas
        this->inputs.resize(inputs);
        this->hidden.resize(hidden);
        this->output.resize(outputs);
        this->target.resize(outputs);
        this->hidden_delta.resize(hidden);
        this->output_delta.resize(outputs);

        // Inicializa pesos e bias com valores aleatórios
        srand((unsigned)time(0));
        weights_input_hidden.resize(inputSize, std::vector<double>(hiddenSize));
        weights_hidden_output.resize(hiddenSize, std::vector<double>(outputSize));
        bias_hidden.resize(hiddenSize);
        bias_output.resize(outputSize);

        for (int i = 0; i < inputSize; ++i)
            for (int j = 0; j < hiddenSize; ++j)
                weights_input_hidden[i][j] = ((double) rand() / (RAND_MAX)) - 0.5;

        for (int i = 0; i < hiddenSize; ++i)
            for (int j = 0; j < outputSize; ++j)
                weights_hidden_output[i][j] = ((double) rand() / (RAND_MAX)) - 0.5;

        for (int i = 0; i < hiddenSize; ++i)
            bias_hidden[i] = ((double) rand() / (RAND_MAX)) - 0.5;

        for (int i = 0; i < outputSize; ++i)
            bias_output[i] = ((double) rand() / (RAND_MAX)) - 0.5;
    }

    // Método para a propagação direta (forward propagation)
    void forward(std::vector<double>& inputs) {
        this->inputs = inputs;
        // Ativação dos neurônios da camada oculta
        for (int i = 0; i < hiddenSize; ++i) {
            hidden[i] = 0;
            for (int j = 0; j < inputSize; ++j) {
                hidden[i] += this->inputs[j] * weights_input_hidden[j][i];
            }
            hidden[i] += bias_hidden[i];
            hidden[i] = sigmoid(hidden[i]);
        }

        // Ativação dos neurônios da camada de saída
        for (int i = 0; i < outputSize; ++i) {
            output[i] = 0;
            for (int j = 0; j < hiddenSize; ++j) {
                output[i] += hidden[j] * weights_hidden_output[j][i];
            }
            output[i] += bias_output[i];
        }
    }
};
```

```

        output[i] = sigmoid(output[i]);
    }
}

// Método para a retropropagação (backpropagation)
void backpropagate(std::vector<double>& targets, double learnRate) {
    // Calcular o delta para a camada de saída
    for (int i = 0; i < outputSize; ++i) {
        double error = targets[i] - output[i];
        output_delta[i] = error * sigmoid_derivative(output[i]);
    }

    // Calcular o delta para a camada oculta
    for (int i = 0; i < hiddenSize; ++i) {
        double error = 0.0;
        for (int j = 0; j < outputSize; ++j) {
            error += output_delta[j] * weights_hidden_output[i][j];
        }
        hidden_delta[i] = error * sigmoid_derivative(hidden[i]);
    }

    // Atualizar os pesos e bias para a camada de saída
    for (int i = 0; i < hiddenSize; ++i) {
        for (int j = 0; j < outputSize; ++j) {
            weights_hidden_output[i][j] += learnRate * output_delta[j] *
hidden[i];
        }
    }

    // Atualizar os pesos e bias para a camada oculta
    for (int i = 0; i < inputSize; ++i) {
        for (int j = 0; j < hiddenSize; ++j) {
            weights_input_hidden[i][j] += learnRate * hidden_delta[j] *
inputs[i];
        }
    }

    for (int i = 0; i < outputSize; ++i) {
        bias_output[i] += learnRate * output_delta[i];
    }

    for (int i = 0; i < hiddenSize; ++i) {
        bias_hidden[i] += learnRate * hidden_delta[i];
    }
}

};

int main() {
    NeuralNetwork nn(3, 5, 2); // Rede com 3 entradas, 5 neurônios na camada escondida,
e 2 saídas
    std::vector<double> inputs = {1.0, 0.5, -1.0};
    std::vector<double> targets = {0.0, 1.0}; // Valores alvo para treinamento
    nn.forward(inputs);
    nn.backpropagate(targets, 0.1); // Treinamento com taxa de aprendizado de 0.1
    return 0;
}

```

Código-fonte 7 – Código completo da implementação de uma rede neural

Fonte: elaborado pelo autor (2024)

O código implementado em C++ descreve uma rede neural com uma arquitetura de múltiplas camadas que inclui uma camada de entrada, uma camada oculta e uma camada de saída. A camada de entrada possui três neurônios, correspondendo ao número de características dos dados de entrada. A camada oculta

é composta por cinco neurônios, o que permite que a rede capture padrões mais complexos nos dados. A camada de saída, com dois neurônios, indica que a rede pode estar configurada para resolver problemas de classificação binária ou fornecer duas saídas contínuas para problemas de regressão.

Cada neurônio nas camadas oculta e de saída está equipado com um conjunto de pesos e um bias, que são inicializados com valores aleatórios. Estes parâmetros são fundamentais para a transformação dos dados de entrada em saídas úteis. Durante a propagação direta ou “forward propagation”, os dados são processados começando pela camada de entrada, passando pela camada oculta e finalmente chegando à camada de saída.

Em cada camada, as entradas são combinadas linearmente com os respectivos pesos e bias e o resultado é passado por meio de uma função de ativação sigmoid. Esta função é crucial por introduzir não-linearidade ao modelo, permitindo que a rede aprenda relações complexas entre as características de entrada e as saídas desejadas.

Após a obtenção das saídas da rede, o processo de backpropagation é utilizado para ajustar os pesos e vieses. Esse método emprega o cálculo do gradiente da função de custo, que mede o erro entre as saídas preditas e os valores alvo. Esse erro é, então, usado para fazer ajustes nos parâmetros da rede de maneira que o erro seja minimizado nas futuras previsões. Os ajustes são feitos em proporção definida pela taxa de aprendizado, que determina o tamanho dos passos no espaço de parâmetros durante o treinamento.

Este modelo de rede neural é um exemplo prático que ilustra não apenas a configuração inicial de uma rede neural profunda, mas também como ela aprende a partir dos dados por meio do ajuste iterativo de seus parâmetros internos. Este processo de aprendizado iterativo permite que a rede melhore continuamente seu desempenho na tarefa específica para a qual foi treinada, seja ela classificação ou regressão.

Ao utilizar uma combinação de forward propagation e backpropagation, a rede pode adaptar seus parâmetros internos para capturar com precisão as complexidades dos dados e realizar previsões precisas, destacando a capacidade das redes neurais de lidar com problemas complexos em aprendizado de máquina e inteligência artificial.

Em nosso estudo sobre as bases matemáticas das redes neurais, exploramos profundamente a teoria de aprendizado que sustenta essas tecnologias poderosas, conforme descrito no primeiro capítulo de "Mathematical Aspects of Deep Learning" (2023). Nossa jornada começa com a compreensão dos espaços de entrada e saída, que são fundamentais para definir como os dados são processados e interpretados pelas redes neurais. Esses conceitos são essenciais, pois estabelecem o terreno para a definição de hipóteses e funções de perda, que são cruciais para o desenvolvimento de modelos precisos e eficazes.

À medida que avançamos, consideramos a importância da profundidade das redes neurais. A discussão é enriquecida pelo entendimento de que redes mais profundas têm uma capacidade única de representar funções complexas muito mais eficientemente do que redes mais superficiais. Esta capacidade é especialmente significativa quando consideramos funções radiais e redes que utilizam a função de ativação ReLU, um tema amplamente abordado no livro.

Outro aspecto fundamental que exploramos é como as redes neurais podem superar a maldição da dimensionalidade. Isso é alcançado por meio da suposição de que os dados de alta dimensão tendem a residir em variedades matemáticas de baixa dimensão. Essa suposição permite que as redes neurais aprendam eficientemente mesmo com a alta dimensionalidade dos dados, aproveitando estruturas subjacentes menos complexas que são inerentes aos dados.

A otimização das redes neurais também recebe uma atenção detalhada. Analisamos o landscape de perda dessas redes, discutindo como características como mínimos locais e pontos de sela podem afetar o treinamento. A discussão se aprofunda na descrição de como o gradiente descendente e outras estratégias de otimização, como o treinamento preguiçoso, desempenham papéis cruciais na eficácia do aprendizado de máquina profundo, conforme detalhado no livro.

Por fim, dedicamos tempo para entender as arquiteturas especiais de redes neurais, como as redes convolucionais e recorrentes. Essas arquiteturas são projetadas para melhorar a capacidade de aprendizado e generalização dos modelos e são particularmente eficazes em tarefas de visão computacional e processamento sequencial.

Cada um desses tópicos é embasado por definições rigorosas e teoremas matemáticos que fornecem uma compreensão clara e quantificável dos princípios que regem o funcionamento das redes neurais. Por exemplo, a arquitetura de uma rede neural profunda é ilustrada e discutida em detalhes na figura 1, oferecendo uma visualização clara de como as camadas interagem e contribuem para a complexidade do modelo.

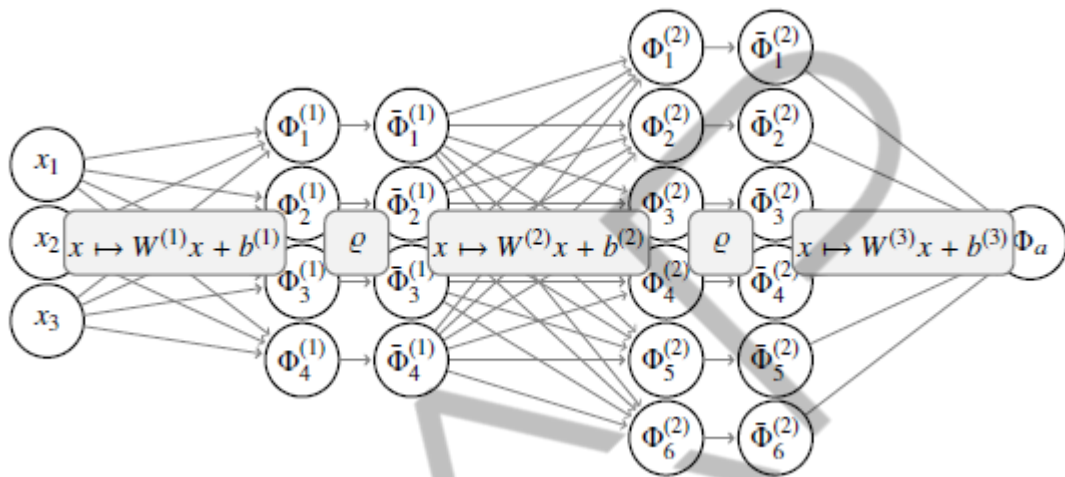


Figura 1 – Grafo e ativações dos neurônios de uma rede neural profunda
Fonte: Grohs; Kutyniok (2023)

A exploração desses conceitos matemáticos não apenas enriquece nosso entendimento teórico sobre redes neurais, como também nos prepara melhor para aplicar esses conhecimentos em problemas práticos de aprendizado de máquina. Aprofundando-se nos fundamentos matemáticos, estamos mais equipados para enfrentar desafios mais complexos e inovar no campo da inteligência artificial.

Fundamentos da Teoria da Aprendizagem e Otimização

Para compreender as bases matemáticas das redes neurais, vamos explorar o conteúdo do primeiro capítulo do livro "Mathematical Aspects of Deep Learning" (2023), em que são apresentados definições e teoremas fundamentais. Vamos abordar esses conceitos e explicar como eles se encaixam nas redes neurais profundas.

Definição 1.1 (Aprendizado - informal)

Sejam $\mathcal{X}$, $\mathcal{Y}$ e $\mathcal{Z}$ espaços mensuráveis. Em uma tarefa de aprendizagem, é dado um conjunto de informação (dados) em $\mathcal{Z}$ e uma função de perda $L: M(\mathcal{X}, \mathcal{Y}) \times \mathcal{Z} \rightarrow \mathbb{R}$. O objetivo é escolher um conjunto de hipóteses $F \subset M(\mathcal{X}, \mathcal{Y})$ e construir um algoritmo de aprendizagem, i.e. um mapa $A: \bigcup_{m \in \mathbb{N}} \mathcal{Z}^m \rightarrow F$, que usa os dados de treinamento $s = (z^i)_{i=1}^m \in \mathcal{Z}^m$ para encontrar o modelo $f_s = A(s) \in F$ que performe bem no conjunto de treinamento s e também generalize para dados não observados $z \in \mathcal{Z}$.

Aqui, o desempenho é medido por meio da função de perda L e da perda correspondente $L(f_s, z)$, e, informalmente, a generalização significa que o desempenho fora da amostra de f_s em z se comporta de forma similar ao desempenho na amostra s .

O entendimento do tipo de problema de aprendizado começa com a definição de espaços de entrada $\mathcal{X}$ e saída $\mathcal{Y}$, que são fundamentais para definir como os dados são processados e interpretados pelas redes neurais. Esses conceitos estabelecem o terreno para a definição de hipóteses e funções de perda, cruciais para o desenvolvimento de modelos precisos e eficazes. A função de perda mede quão bem a saída da rede se alinha com os valores esperados, sendo minimizada ajustando os pesos da rede.

Definição 1.2 (Tarefa de previsão)

Em uma tarefa de previsão, temos $Z \subset X \times Y$, ou seja, recebemos dados de treinamento $s = (x^i, y^i)_{i=1}^m$ que consistem em características de entrada $x^i \in X$ e rótulos correspondentes $y^i \in Y$. Para tarefas de regressão unidimensional com $Y \subset \mathbb{R}$, consideramos a perda quadrática $L(f, (x, y)) = (f(x) - y)^2$ e, para tarefas de classificação binária com $Y = \{-1, 1\}$, consideramos a perda 0-1 $L(f, (x, y)) = 1_{(-\infty, 0)}(yf(x))$.

Assumimos que as características de entrada estão no espaço euclidiano, ou seja, $X \subset \mathbb{R}^d$ com dimensão de entrada $d \in \mathbb{N}$. Com essa definição, identificamos como os dados de entrada e saída são utilizados para treinar a rede. A tarefa é encontrar um modelo $f_s : X \rightarrow Y$ que preveja bem os rótulos y a partir de novas entradas x .

Definição 1.3 (Rede neural totalmente conectada - FC)

Uma rede neural totalmente conectada feed-forward é dada por sua arquitetura $a = (N, \rho)$, em que $L \in \mathbb{N}$, $N \in \mathbb{N}^{L+1}$ e $\rho : \mathbb{R} \rightarrow \mathbb{R}$. Referimo-nos a ρ como a função de ativação, a L como o número de camadas e a N_0, N_L , e $N_\ell, \ell \in [L - 1]$, como o número de neurônios na entrada, saída e ℓ -ésima camada oculta, respectivamente. Definimos a função de realização correspondente $\Phi_a : \mathbb{R}^{N_0} \times \mathbb{R}^{P(N)} \rightarrow \mathbb{R}^{N_L}$, que satisfaz para cada entrada $x \in \mathbb{R}^{N_0}$ e parâmetros

$$\theta = (\theta^\ell)_{\ell=1}^L = (W^\ell, b^\ell)_{\ell=1}^L \in \bigoplus_{\ell=1}^L (\mathbb{R}^{N_\ell \times N_{\ell-1}} \times \mathbb{R}^{N_\ell}),$$

$$\Phi_a(x, \theta) = \Phi^{(L)}(x, \theta),$$

onde

$$\Phi^{(1)}(x, \theta) = W^{(1)}x + b^{(1)},$$

$$\bar{\Phi}^{(\ell)}(x, \theta) = \rho(\Phi^{(\ell)}(x, \theta))W^{(\ell+1)}, \ell \in [L - 1] \text{ e}$$

$$\Phi^{(\ell+1)}(x, \theta) = W^{(\ell+1)}\rho(\Phi^{(\ell)}(x, \theta)) + b^{(\ell+1)}, \ell \in [L - 1],$$

e ρ é aplicada componente a componente. Nos referimos a $W^\ell \in \mathbb{R}^{N_\ell \times N_{\ell-1}}$ e $b^\ell \in \mathbb{R}^{N_\ell}$ por matrizes de pesos e vetores de vies e a $\bar{\Phi}^{(\ell)}$ e $\Phi^{(\ell)}$ como funções de ativação e pré-ativação do neurônio N_ℓ da ℓ -ésima camada. Com a definição da rede neural totalmente conectada, compreende-se como a estrutura básica da rede é formada, incluindo camadas de neurônios, pesos e vieses. As funções de ativação, como ReLU e sigmoidal, introduzem não linearidade, permitindo que a rede aprenda representações complexas.

As definições de espaços de entrada X e saída Y estabelecem uma base crucial, tratando o aprendizado de máquina como um problema de mapeamento de um espaço para outro. Esta estrutura ajuda a entender que o objetivo principal das redes neurais é minimizar a função de perda L , que mede o quão bem as saídas da rede alinham-se com os valores alvo esperados. A função de perda é central para o treinamento supervisionado e define claramente a tarefa da rede neural.

A introdução de uma função de hipótese dentro de um conjunto de funções possíveis F e a minimização de uma função de perda associada ilustram o processo de aprendizado, em que a rede neural ajusta seus parâmetros para reduzir a discrepância entre as previsões e os valores reais. A escolha da função de perda, seja

ela quadrática ou logística, tem um impacto direto na aprendizagem e na capacidade de generalização da rede, influenciando a maneira como ela responde a variações nos dados de entrada.

Além disso, a teoria de margem e o regime de kernel são discutidos para explicar como as redes neurais podem ser avaliadas e otimizadas. A teoria de margem revela que redes com grandes margens de classificação tendem a ter melhor desempenho e menor sobreajuste (overfitting), enquanto o regime de kernel mostra como as redes podem efetivamente capturar interações complexas nos dados por meio de transformações em espaços de características de alta dimensão.

O teorema do "No free lunch" (teorema 1.14 do livro "Mathematical Aspects of Deep Learning" (2023)), ressalta que não existe um único algoritmo de aprendizado superior para todas as tarefas e distribuições de dados. Isso sublinha a importância de escolher e adaptar modelos de redes neurais de acordo com as características específicas de cada conjunto de dados. Essa compreensão enfatiza a necessidade de adaptabilidade e personalização no design de redes neurais.

Ainda, a aplicação prática desses conceitos é exemplificada pelo Algoritmo de Gradiente Estocástico Descendente (SGD), que ajusta os parâmetros das redes neurais de forma iterativa para minimizar a função de perda. A eficácia do SGD, combinada com a escolha adequada do tamanho do passo e métodos de cálculo do gradiente, é crítica para o treinamento eficiente das redes, impactando diretamente a velocidade de convergência e a qualidade do modelo final.

Inferência Bayesiana e Espaços Latentes

Explorar a Regra de Bayes e suas aplicações em aprendizado supervisionado e não supervisionado é fundamental para compreender a vasta aplicabilidade e influência desta fórmula na ciência de dados e aprendizado de máquina. A **Regra de Bayes** é uma expressão matemática que quantifica a probabilidade de uma hipótese com base em evidências anteriores e novas informações. Matematicamente, é expressa como $P(H|E) = \frac{P(E|H) P(H)}{P(E)}$, sendo que $P(H|E)$ é a probabilidade da hipótese H dado o evento E , $P(E|H)$ é a probabilidade de observar o evento E dado que a hipótese H é verdadeira, $P(H)$ é a probabilidade a priori da hipótese H e $P(E)$ é a probabilidade total do evento E .

No contexto de aprendizado supervisionado, a Regra de Bayes é usada para atualizar o conhecimento sobre os parâmetros do modelo à medida que novos dados são observados. Por exemplo: em classificação, pode-se usar essa abordagem para calcular a probabilidade posterior de diferentes classes com base em características observadas dos dados, permitindo assim uma modelagem preditiva mais precisa. Isso é especialmente útil em cenários nos quais a distribuição de classes é desigual ou quando os dados são susceptíveis a variações devido a condições externas.

Em aprendizado não supervisionado, a Regra de Bayes ajuda na identificação de estruturas ocultas ou padrões nos dados. Por exemplo: algoritmos como a análise de componentes principais (PCA) e a alocação de Dirichlet latente (LDA) utilizam princípios bayesianos para inferir a distribuição dos dados em espaços de menor dimensão, conhecidos como espaços latentes. Esses espaços latentes facilitam a compressão e visualização dos dados, assim como a identificação de grupos ou clusters naturais dentro dos dados.

A capacidade da Regra de Bayes de incorporar tanto conhecimento prévio quanto novas informações faz dela uma ferramenta poderosa para a modelagem estatística. Isso é evidenciado na sua aplicação em filtros de Kalman e filtros de partículas, que são amplamente usados em navegação e rastreamento, em que estimativas contínuas precisam ser atualizadas em tempo real.

Os filtros de Kalman são uma técnica poderosa para estimar o estado de um sistema linear dinâmico a partir de uma série de medições ao longo do tempo, que são ruidosas ou incompletas. Eles são amplamente usados em aplicações de rastreamento e navegação, como o controle de veículos autônomos e aeronaves, e em qualquer contexto em que seja necessário estimar continuamente o estado de um sistema em movimento ou em mudança. Formalmente, o filtro de Kalman assume que o estado do sistema evolui de acordo com uma equação linear de primeira ordem:

$$x_k = F_k x_{k-1} + B_k u_k + w_k$$

em que:

- x_k é o vetor de estado no tempo k .
- F_k é a matriz de transição de estado, que aplica o efeito do estado anterior x_{k-1} no estado atual.

- B_k é a matriz de controle que aplica o efeito da entrada de controle u_k .
- w_k é o processo de ruído, assumido como gaussiano com média zero e covariância Q_k .

As medições z_k , que são observações que podem ser usadas para estimar o estado, são modeladas como:

$$z_k = H_k x_k + v_k$$

em que:

- H_k é a matriz que mapeia o estado verdadeiro para as medições observadas.
- v_k é o ruído de medição, também assumido como gaussiano com média zero e covariância R_k .

O objetivo do filtro de Kalman é usar essas medições para produzir uma estimativa do estado que seja tão precisa quanto possível. O filtro opera em dois passos: predição e atualização.

1. Predição:

- Estima o estado atual com base no estado estimado e na entrada de controle do passo anterior.
- Calcula a covariância do erro de predição.

2. Atualização:

- Usa a nova medição z_k para atualizar a estimativa do estado.
- Ajusta a covariância do erro com base na medição atual.

A atualização é onde a Regra de Bayes entra em jogo. O filtro de Kalman utiliza essa regra para combinar a estimativa predita (prior) com a nova medição (evidência) para obter uma estimativa atualizada (posterior), que é mais informada. Essencialmente, a Regra de Bayes ajuda a fundir as informações novas e antigas de maneira probabilisticamente fundamentada, ajustando as estimativas com base na incerteza de cada fonte de informação.

Os filtros de Kalman estão intrinsecamente relacionados ao conceito de espaços latentes porque o vetor de estado x_k pode ser visto como uma representação

latente do sistema físico. Esta representação captura as características essenciais do sistema que são necessárias para prever o comportamento futuro e não são diretamente observáveis. Portanto, o filtro de Kalman serve como uma excelente ilustração teórica da importância dos espaços latentes quando discutimos conceitos de probabilidade condicional.

A Regra de Bayes também permite sua aplicação em cenários complexos em que múltiplas variáveis estão inter-relacionadas. Isso é particularmente útil em campos como genética, no qual a expressão de genes pode ser influenciada por múltiplos fatores, ou em sistemas de recomendação, em que a preferência do usuário pode ser inferida a partir de uma combinação de suas interações passadas e atributos dos itens.

Espaços latentes referem-se a representações ocultas ou abstratas em modelos de aprendizado de máquina que capturam informações essenciais dos dados que não são diretamente observáveis. Estes espaços são fundamentais em muitos modelos de aprendizado profundo, especialmente em algoritmos generativos, em que ajudam a modelar a distribuição dos dados de maneira compacta e eficiente.

Explorar esses espaços latentes é crucial para entender não apenas a estrutura subjacente dos dados, mas também para realizar inferências sobre novos dados com base nesses modelos aprendidos. Portanto, a Regra de Bayes, ao oferecer uma maneira sistemática de atualizar as crenças em face de novas evidências, prova ser uma abordagem indispensável em muitos aspectos do aprendizado de máquina moderno, tanto supervisionado quanto não supervisionado.

Formalmente, um espaço latente pode ser definido como um espaço de menor dimensão em que os dados de alta dimensão são projetados. Matematicamente, isso é frequentemente representado por uma função de mapeamento $f: \mathbb{R}^d \rightarrow \mathbb{R}^k$, em que d é a dimensão original dos dados e k é a dimensão do espaço latente, com $k < d$. Este mapeamento visa preservar alguma estrutura essencial dos dados originais, como distâncias ou variações internas, enquanto reduz a quantidade de ruído ou redundância.

O processo de aprender essas representações latentes envolve a descoberta de uma função que mapeia os dados de entrada para um novo espaço no qual os aspectos significativos são mais facilmente analisáveis ou interpretáveis. Em contextos práticos, isso significa identificar padrões, clusters ou componentes

principais que explicam a maior parte da variância nos dados, sem necessariamente reter toda a complexidade do conjunto de dados original.

Por exemplo: na Análise de Componentes Principais (PCA), a transformação linear é usada para reduzir a dimensionalidade dos dados, projetando-os nas direções que maximizam a variância. Outro exemplo é a modelagem de tópicos em processamento de linguagem natural, em que técnicas como Alocação Latente de Dirichlet (LDA) identificam tópicos como distribuições sobre um conjunto fixo de palavras e cada documento é representado como uma mistura desses tópicos.

Aqui, o espaço latente é composto pelas distribuições de tópicos e o processo de inferência envolve determinar quais tópicos estão presentes em um documento dado, proporcionando uma compreensão mais profunda da estrutura semântica dos dados.

Em redes neurais profundas, especialmente em autoencoders, o espaço latente é formado pela camada central da rede, que condensa a informação de entrada em uma forma comprimida. A rede aprende a codificar a entrada para esse espaço latente e depois decodificar de volta para a reconstrução da entrada original, facilitando tarefas como redução de dimensionalidade, denoising e geração de novas instâncias de dados que são semelhantes aos dados de treinamento.

Além disso, a Regra de Bayes desempenha um papel crucial na interpretação e utilização de espaços latentes, especialmente em modelos probabilísticos generativos. Ao modelar as variáveis latentes como variáveis aleatórias, a inferência Bayesiana pode ser usada para estimar as distribuições dessas variáveis com base nos dados observados. Isso permite uma abordagem rigorosa para lidar com incertezas e inferir informações que não são diretamente acessíveis por meio dos dados observados.

Tensores

O conceito de espaços latentes é fundamental para o entendimento do funcionamento de redes neurais, pois habilita a existência de redes neurais generativas. No entanto, existem conceitos ainda mais fundamentais, que permeiam todas as representações formais e definições das redes neurais. Dentre estes conceitos, um dos mais fundamentais diz respeito aos tensores.

Explorando os conceitos de tensores e sua importância na álgebra linear e no desenvolvimento de redes neurais profundas, deparamo-nos com uma base matemática essencial para entender as operações subjacentes em muitos algoritmos modernos de aprendizado de máquina. Tensores são entidades matemáticas que generalizam escalares, vetores e matrizes a dimensões superiores e são fundamentais para o desenvolvimento de técnicas e manipulação de dados em formatos complexos como imagens, vídeo e outras formas de dados multidimensionais.

Em termos matemáticos, um tensor pode ser visto como um vetor multidimensional, em que cada elemento é identificado por um conjunto de índices, cada um representando uma dimensão no espaço do tensor. Por exemplo: um tensor de ordem dois é uma matriz em que os elementos são acessados por dois índices. Essa generalização continua para tensores de ordem superior, com três ou mais índices. Matematicamente, um tensor de ordem n em um espaço tridimensional seria representado como T_{ijk} , sendo que i, j, k variam sobre as dimensões do tensor.

Na álgebra linear, tensores são essenciais porque permitem a representação e a manipulação compacta de relações lineares complexas entre vetores, matrizes e outros tensores. Isso é especialmente útil em redes neurais profundas, nas quais a transformação de grandes quantidades de dados através de várias camadas requer uma formulação eficiente e generalizável que tensores fornecem naturalmente. As operações tensoriais como multiplicação tensorial e soma tensorial são blocos de construção para operações de redes neurais, incluindo a propagação para frente e para trás (forward e backward propagation), essenciais para o treinamento de modelos.

A importância dos tensores estende-se além da simples manipulação de dados. Na teoria das redes neurais, eles são fundamentais para entender como as transformações lineares e não lineares são aplicadas em escalas multidimensionais. Cada camada de uma rede neural profunda pode ser conceptualizada como uma transformação tensorial que mapeia os dados de entrada, através de pesos representados por tensores, para produzir uma saída desejada, também organizada como um tensor.

A introdução de tensores no campo de aprendizado profundo também facilitou o desenvolvimento de bibliotecas e frameworks que suportam cálculos intensivos em tensores, como TensorFlow e PyTorch. Estes frameworks utilizam a potência computacional dos tensores para realizar operações de álgebra linear em grande escala, o que é essencial para o processamento e o treinamento eficiente de modelos de dados grandes.

Além disso, tensores têm uma aplicação direta na melhoria da eficiência computacional e na capacidade de modelos de redes neurais em tarefas de visão computacional, processamento de linguagem natural e outros domínios de aprendizado de máquina. Por exemplo: na visão computacional, tensores permitem a manipulação eficiente de imagens como arrays de três dimensões (altura, largura, canais de cor), facilitando operações como convoluções e pooling, que são fundamentais para redes convolucionais modernas.

A compreensão de tensores e sua manipulação na álgebra linear é vital para qualquer pessoa trabalhando com deep learning. Ela não apenas apoia a implementação de redes neurais, mas também proporciona uma compreensão mais profunda de como os dados são transformados em complexas arquiteturas de rede, permitindo inovações continuadas em design de modelo e otimização de algoritmos.

Para um entendimento formal da estrutura de tensores e suas aplicações às redes neurais convolucionais e recorrentes, consulte o capítulo 05 do livro “Mathematical Aspects of Deep Learning”, que apresenta uma explicação profunda da formulação de tensores a partir da teoria quântica.

Funções de Ativação

Associando ao conceito de tensores, é crucial entender como as funções de ativação operam dentro das redes neurais profundas, transformando os tensores de entrada em tensores de saída em cada camada. Essa transformação é fundamental para adicionar complexidade e capacidade de modelagem às redes, permitindo-lhes aprender e representar uma vasta gama de funções não lineares que são críticas para tarefas como classificação, regressão e mais.

As funções de ativação são fundamentais para permitir que essas redes aprendam e modelem complexidades não-lineares em dados variados. Funções de ativação são aplicadas a cada neurônio da rede neural e decidem se ele deve ser

ativado ou não, isto é, enviar seu sinal adiante na rede. A escolha da função de ativação impacta significativamente a capacidade da rede de convergir durante o treinamento e de aproximar funções complexas.

As funções de ativação são diferenciáveis e não lineares. Formalmente, uma função $f: \mathbb{R}^n \rightarrow \mathbb{R}^m$ é dita diferenciável em um ponto $\mathbf{a} \in \mathbb{R}^n$ se existir uma aplicação linear $L: \mathbb{R}^n \rightarrow \mathbb{R}^m$ tal que:

$$\lim_{\mathbf{h} \rightarrow \mathbf{0}} \frac{|f(\mathbf{a} + \mathbf{h}) - f(\mathbf{a}) - L(\mathbf{h})|}{|\mathbf{h}|} = 0$$

Se essa condição é satisfeita, dizemos que L é a derivada de f em $\mathbf{a}$ e pode ser representada matricialmente quando f é de dimensão adequada. Esta definição implica que a função f pode ser aproximada por uma transformação linear nas proximidades de $\mathbf{a}$ e a taxa de variação da função em relação a cada componente de seu domínio pode ser descrita precisamente por L .

Uma função é considerada não linear se ela não pode ser expressa como uma soma ponderada de suas variáveis de entrada sem transformações adicionais, ou seja, se ela não pode ser descrita pela forma $f(\mathbf{x}) = \mathbf{Ax} + \mathbf{b}$ para qualquer matriz $\mathbf{A}$ e vetor $\mathbf{b}$. Matematicamente, isso significa que a função não satisfaz a propriedade da aditividade e da homogeneidade para todas as entradas e escalares, i.e.,

$$f(\mathbf{x} + \mathbf{y}) \neq f(\mathbf{x}) + f(\mathbf{y}) \quad \text{ou} \quad f(a\mathbf{x}) \neq af(\mathbf{x}) \quad \text{para alguns } \mathbf{x}, \mathbf{y} \in \mathbb{R}^n \quad \text{e} \quad a \in \mathbb{R}$$

Essas duas propriedades — diferenciabilidade e não linearidade — são centrais no estudo de redes neurais, em que as funções de ativação introduzem a não linearidade necessária para que os modelos possam aprender padrões complexos nos dados, que não seriam possíveis de captar por meio de modelos puramente lineares. A diferenciabilidade, por outro lado, é crucial porque permite a aplicação do algoritmo de retropropagação (backpropagation), que ajusta os pesos da rede de forma eficiente.

A relação das funções de ativação com a teoria dos tensores e sua aplicação em espaços latentes é igualmente importante. Em redes profundas, os tensores não apenas representam entrada, pesos e saídas, mas também os gradientes das funções de ativação, que são usados durante a retropropagação. A não linearidade introduzida pelas funções de ativação permite que a rede aprenda transformações complexas dos

dados de entrada, efetivamente parametrizando os espaços latentes que são essenciais para tarefas como reconhecimento de imagem e processamento de linguagem natural.

A ativação condicional, mencionada como uma parte da capacidade das funções de ativação de serem diferenciáveis, refere-se à habilidade de uma rede em ajustar seu comportamento com base na entrada recebida. Isso é crucial em modelos que dependem de partes da entrada para tomar decisões, como em redes neurais que processam partes de imagens separadamente antes de tomar uma decisão de classificação.

Associando ao conceito de tensores, é crucial entender como as funções de ativação operam dentro das redes neurais profundas, transformando os tensores de entrada em tensores de saída em cada camada. Essa transformação é fundamental para adicionar complexidade e capacidade de modelagem às redes, permitindo-lhes aprender e representar uma vasta gama de funções não-lineares que são críticas para tarefas como classificação, regressão e mais.

Entre as funções de ativação mais utilizadas, a ReLU (Unidade Linear Retificada) se destaca por sua simplicidade e eficácia, sendo definida como $f(x) = \max_x 0, x$. A ReLU é particularmente popular porque ajuda a mitigar o problema do desvanecimento do gradiente em redes profundas, permitindo que modelos com muitas camadas sejam treinados de forma mais eficiente. Além disso, ao zerar os valores negativos, ela introduz esparsidade na ativação dos neurônios, o que pode ser vantajoso em termos de memória e computação.

Outra função de ativação comum é a sigmoide, $f(x) = \frac{1}{1 + e^{-x}}$, que produz uma saída entre 0 e 1. Isso é útil especialmente em camadas de saída de redes neurais destinadas a tarefas de classificação binária, em que o resultado pode ser interpretado como uma probabilidade. No entanto, a função sigmoide cai em desuso em camadas ocultas devido ao seu gradiente, que pode ser muito pequeno longe da origem, o que leva ao problema do desvanecimento do gradiente.

A função tangente hiperbólica, ou tanh, é outra função clássica, definida como $f(x) = \tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$. Ela é semelhante à sigmoide, mas mapeia os valores de entrada para um intervalo de -1 a 1. Isso a torna mais eficaz que a sigmoide em certas

configurações, pois os valores negativos podem ajudar a centrar os dados de saída, melhorando a convergência durante o treinamento.

Além dessas, a função de ativação Leaky ReLU, uma variação da ReLU, permite que uma pequena quantidade de gradiente seja propagada mesmo para valores de entrada negativos: $f(x) = \max(\alpha x, x)$, em que α é um pequeno coeficiente positivo. Essa propriedade é útil para manter alguma atividade nos neurônios e evitar o problema dos neurônios "mortos", comum na ReLU padrão.

Essas funções de ativação, quando aplicadas dentro de redes neurais profundas, transformam os tensores de entrada em tensores de saída com propriedades desejadas para a próxima camada, formando a base para a complexa arquitetura de aprendizado que caracteriza as redes neurais modernas. Cada função de ativação adiciona uma camada de não linearidade, sem a qual a rede seria apenas uma combinação linear de suas entradas, incapaz de capturar as complexidades dos dados reais.

Portanto, as funções de ativação são mais do que simples transformações; elas são o coração das redes neurais profundas, dando-lhes a capacidade de aprender e modelar a vasta gama de padrões encontrados em dados complexos. Ao ajustar os tensores que fluem através da rede, essas funções permitem que redes profundas capturem e representem complexidades que seriam inacessíveis com métodos lineares, solidificando sua posição como uma das ferramentas mais poderosas no campo da inteligência artificial.

Otimização, Feedforward e Backward propagation

Ao abordar a otimização em redes neurais, exploramos um dos aspectos mais cruciais no treinamento de modelos de aprendizado de máquina. A otimização em redes neurais envolve ajustar os parâmetros do modelo, como os pesos das conexões e os vieses, de modo a minimizar uma função de custo, que geralmente representa o erro entre as previsões do modelo e os valores reais dos dados.

O algoritmo de gradiente descendente estocástico (SGD) é uma ferramenta fundamental para essa otimização. Formalmente, o SGD atualiza os parâmetros θ do modelo iterativamente com o objetivo de minimizar a função de custo $J(\theta)$, que depende do conjunto de dados e dos parâmetros do modelo. A atualização em cada etapa t é feita de acordo com a regra:

$$\theta_{t+1} = \theta_t - \eta \nabla J(\theta_t)$$

em que η é a taxa de aprendizado e $\nabla J(\theta_t)$ é o gradiente da função de custo em relação aos parâmetros no passo t . Em contraste com o gradiente descendente tradicional, que computa o gradiente usando todo o conjunto de dados, o SGD estima o gradiente a partir de um subconjunto aleatório dos dados, chamado de minibatch. Isso torna o SGD particularmente eficaz para conjuntos de dados grandes, reduzindo significativamente a carga computacional e a memória necessária por iteração.

A normalização dos dados e dos gradientes durante o processo de otimização é outra técnica crítica que melhora a eficácia do treinamento. A normalização dos dados de entrada garante que todas as características contribuam igualmente para a função de custo, evitando que características com maior magnitude dominem o processo de aprendizagem. Isso é geralmente alcançado subtraindo a média e dividindo pelo desvio padrão de cada característica.

Além disso, técnicas como o gradiente descendente normalizado (NGD) e a normalização em lote (batch normalization) são usadas para ajustar e estabilizar o treinamento de redes neurais. O NGD ajusta cada passo do gradiente de forma que a magnitude do passo seja constante, independentemente da escala dos gradientes. Isso é particularmente útil em cenários nos quais o gradiente pode variar significativamente em magnitude, o que pode levar a passos de atualização muito grandes ou muito pequenos.

A normalização em lote, por outro lado, ajusta as ativações internas da rede para terem média zero e variância unitária ao longo de cada minibatch. Isso ajuda a evitar o problema do desaparecimento ou explosão dos gradientes durante o treinamento, especialmente em redes profundas, garantindo que os gradientes sejam estáveis e facilitando o uso de taxas de aprendizado mais altas.

Essas técnicas de otimização e normalização são essenciais para treinar efetivamente redes neurais profundas, que podem ter muitas camadas e milhões de parâmetros. Sem essas técnicas, o treinamento de tais modelos seria impraticável, pois os gradientes se tornariam muito difíceis de controlar, levando a convergências lentas ou mesmo falhas no treinamento.

No contexto de otimização de redes neurais, é essencial entender como o backpropagation, uma técnica fundamental, facilita o ajuste sistemático dos pesos da

rede para minimizar a função de custo. Em redes neurais feedforward, em que o fluxo de dados se move estritamente da entrada para a saída sem loops, o backpropagation serve como um método eficaz para calcular o gradiente da função de custo em relação a cada peso da rede.

Este cálculo é crucial para a aplicação do algoritmo de gradiente descendente estocástico (SGD), permitindo a atualização dos pesos de modo que o erro entre as saídas preditas e os valores reais seja progressivamente reduzido. A integração do backpropagation no processo de otimização não só maximiza a eficiência do treinamento como também aprimora a capacidade do modelo de aprender representações complexas dos dados, que são fundamentais para o desempenho das redes em tarefas variadas.

O backpropagation, ou propagação reversa do erro, é um método que calcula o gradiente da função de custo (ou função de erro) de uma rede neural em relação a cada peso. Ele é usado para implementar a descida do gradiente, atualizando os pesos para minimizar o erro. O processo começa na camada de saída e propaga-se para trás, daí o nome "backpropagation".

Matematicamente, considerando $J(\theta)$ como a função de custo da rede, em que θ simboliza todos os pesos e bias, o objetivo é calcular o gradiente $\nabla J(\theta)$. Para um neurônio na camada de saída, o gradiente do erro em relação ao seu peso θ é calculado como $\partial J / \partial \theta = \partial J / \partial a \cdot \partial a / \partial \theta$, sendo que a é a ativação do neurônio. Esse gradiente reflete como uma pequena mudança no peso afeta a função de custo, que é crítico para ajustar o peso de maneira a reduzir o erro.

À medida que o erro é propagado para trás, para pesos em camadas mais internas, a computação do gradiente envolve não apenas as derivadas da função de custo em relação às ativações dessas camadas, mas também o impacto dessas ativações nas subsequentes. Para um peso específico θ que conecta neurônios de duas camadas consecutivas, o cálculo envolve o gradiente do erro em relação à ativação da camada seguinte, a derivada da ativação em relação à sua entrada linear e finalmente a derivada dessa entrada em relação ao peso em questão. Este processo é repetido iterativamente, refletindo como o ajuste de um peso em uma camada mais profunda influencia o erro na saída final da rede.

Cada peso é então ajustado subtraindo um produto da taxa de aprendizado η pelo gradiente calculado. Esse ajuste é feito de forma que minimize a função de custo $J(\theta)$, levando a uma melhora progressiva na precisão das previsões feitas pela rede.

Para uma rede feedforward, o fluxo de dados ocorre inicialmente na fase de propagação direta (feedforward phase), em que as entradas são processadas pelas camadas ocultas até gerarem uma saída. Durante essa fase, cada neurônio recebe um conjunto de entradas, aplica uma função de ativação e passa o resultado para frente. As funções de ativação, como ReLU ou sigmoid, introduzem não linearidades essenciais para aprender padrões complexos.

Após a propagação direta, se a saída da rede não corresponde ao esperado, o erro é calculado, geralmente como a diferença entre a saída esperada e a saída produzida pela rede. Este erro é então utilizado para iniciar o backpropagation: ele é propagado de volta pela rede, camada por camada, calculando-se o gradiente do erro em relação a cada peso pelo caminho. Esse gradiente informa como os pesos devem ser ajustados para minimizar o erro.

O cálculo do gradiente em cada camada é realizado utilizando a regra da cadeia do cálculo diferencial. Para um peso específico, o gradiente do erro é proporcional à taxa de mudança do erro em relação à saída do neurônio e à taxa de mudança da saída do neurônio em relação ao peso. A atualização dos pesos é feita subtraindo uma fração do gradiente do peso atual, em que essa fração é determinada pela taxa de aprendizado, um parâmetro crucial que define o tamanho dos passos na descida do gradiente.

A eficiência do backpropagation, particularmente em redes profundas, pode ser aumentada com técnicas como a normalização em lote (batch normalization), que normaliza as saídas de cada camada para ter média zero e variância unitária, estabilizando o processo de aprendizado e permitindo o uso de taxas de aprendizado maiores.

O backpropagation é, portanto, um método poderoso que permite que as redes neurais aprendam de maneira eficiente, ajustando os pesos de forma que o erro seja minimizado em todo o conjunto de treinamento. Isso é essencial para o desenvolvimento de redes neurais que são capazes de realizar tarefas de classificação, regressão e mais com alta precisão. A capacidade das redes

feedforward de lidar com uma ampla gama de tarefas faz delas uma ferramenta versátil e poderosa na caixa de ferramentas de aprendizado de máquina.

Regularização

Explorando a regularização em redes neurais, compreendemos como ajustes específicos podem significativamente melhorar o desempenho dos modelos ao reduzir o overfitting e generalizar melhor para dados não vistos. A regularização é um aspecto crucial na otimização de redes neurais, servindo para controlar a complexidade do modelo e manter o equilíbrio entre o viés e a variância, uma dicotomia central em aprendizado de máquina.

Um dos métodos mais comuns de regularização é o ajuste de viés em cada camada da rede. Ajustar o viés permite que o modelo seja mais flexível e adaptável a diferentes formas de dados, sem necessariamente aumentar o número de parâmetros treináveis. Isso ajuda a evitar o overfitting, garantindo que o modelo não se ajuste demais aos ruídos ou detalhes específicos do conjunto de treinamento.

A adição de termos de regularização, como a penalidade L2, conhecida também como decaimento de peso, é uma estratégia eficaz para impor uma restrição nos pesos da rede. Ao penalizar os pesos grandes, a regularização L2 incentiva a rede a manter os pesos pequenos, levando a um modelo mais simples e, potencialmente, mais generalizável. Matematicamente, isso é representado adicionando um termo proporcional à soma dos quadrados dos pesos à função de custo que o modelo tenta minimizar.

Além disso, o conceito de viés indutivo é fundamental para entender como diferentes escolhas de regularização podem influenciar o desempenho do modelo. Baseando-se no teorema do "no free lunch", que afirma que nenhum algoritmo de aprendizado é universalmente superior a outro se considerarmos todos os possíveis problemas de aprendizado, o viés indutivo de um modelo de aprendizado define a preferência do modelo por uma solução específica.

Em termos práticos, isso significa que a escolha do tipo de regularização pode determinar como o modelo generalizará para novos dados, influenciando diretamente a sua eficácia em tarefas específicas.

Nesse contexto, entender e aplicar técnicas de regularização é uma habilidade essencial para quem trabalha com redes neurais. Por exemplo: em aplicações

supervisionadas e não supervisionadas, a escolha apropriada de regularização pode significar a diferença entre um modelo que performa bem apenas em dados de treinamento e um que é robusto o suficiente para ser aplicado em cenários reais, diversos e variados.

Por meio da regularização, também exploramos métodos para definir melhor os resultados e capacidades das redes, permitindo que os modelos não só aprendam padrões dos dados, mas também incorporem conhecimento a priori ou restrições específicas do problema em questão. Isso é especialmente relevante em áreas nas quais os dados são escassos ou muito ruidosos, em que a regularização pode ajudar a extrair informações úteis apesar das limitações dos dados.

Na otimização de redes neurais, além do já mencionado decaimento de peso ou regularização L2, outras funções de regularização como Lasso (L1) e Ridge (L2) desempenham papéis fundamentais na modelagem e na generalização dos modelos. Essas técnicas são incorporadas para controlar a complexidade do modelo, ajudando a prevenir o overfitting e a promover a capacidade de generalização, essencial para o desempenho robusto em dados não vistos.

A regularização Ridge, ou L2, como já explorado, adiciona um termo de penalidade que é proporcional ao quadrado dos valores dos pesos. Matematicamente, isto é representado pela adição do termo $\lambda \sum_i w_i^2$ à função de custo, em que w_i são os pesos e λ é o parâmetro de regularização que controla a força da penalidade. A principal característica da regularização Ridge é que ela tende a distribuir o erro entre todos os termos, favorecendo um modelo com muitos pesos pequenos, o que pode melhorar a robustez e a generalização do modelo.

Por outro lado, a regularização Lasso, ou L1, adiciona um termo de penalidade que é proporcional ao valor absoluto dos pesos, representado por $\lambda \sum_i |w_i|$. A principal consequência da regularização Lasso é que ela pode resultar em modelos com muitos pesos exatamente iguais a zero, promovendo a esparsidade no modelo final. Isso significa que a Lasso pode não apenas ajudar a reduzir o overfitting, mas também auxiliar na seleção de características, identificando e descartando as variáveis menos importantes para a predição.

A noção de equivariância também é fundamental em contextos de regularização e otimização. Equivariância refere-se à propriedade de uma função que

mantém as transformações aplicadas às entradas na saída. Em termos de redes neurais, isso significa que se a entrada é transformada de uma certa maneira, a saída da rede também deve ser transformada de maneira correspondente. Esta propriedade é crucial para garantir que os modelos aprendam representações que sejam consistentes e previsíveis em resposta a transformações conhecidas dos dados de entrada, como rotações ou translações em imagens.

Portanto, as funções de regularização Lasso e Ridge, juntamente com o conceito de equivariância, são elementos vitais na construção de modelos de rede neural. Eles não apenas ajudam a gerenciar a complexidade e a capacidade do modelo, mas também garantem que as representações aprendidas sejam úteis, relevantes e consistentes em diferentes cenários e transformações de entrada. A escolha adequada e a calibração dessas técnicas de regularização são, portanto, passos cruciais no design de sistemas de aprendizado de máquina eficazes e robustos.

O QUE VOCÊ VIU NESTA AULA?

Em nossa exploração das bases matemáticas das redes neurais, discutimos profundamente a interação entre estruturas de dados complexas e modelos computacionais que permitem máquinas aprenderem a partir de exemplos. O conceito de espaços de entrada e saída foi identificado como crucial, pois define como os dados são tratados e como as informações são transformadas pelas redes neurais durante o processo de aprendizado. Este fundamento teórico é essencial para estabelecer hipóteses e funções de perda que orientam a construção de modelos que não apenas performam bem em dados vistos, mas que também generalizam para novos dados de forma eficaz.

Avançamos na discussão sobre a profundidade das redes neurais, destacando como redes mais profundas são capazes de representar funções complexas de maneira mais eficiente do que as redes menos profundas. Isso é particularmente importante quando consideramos funções radiais e o uso de funções de ativação como a ReLU, que permitem a modelagem de não-linearidades necessárias para captar padrões intrincados nos dados. A profundidade das redes, portanto, não é apenas um detalhe técnico, mas um fator crítico que potencializa a capacidade das redes neurais em executar tarefas de aprendizado de máquina com alta precisão.

Também exploramos como as redes neurais combatem a maldição da dimensionalidade, uma consideração vital quando lidamos com dados de alta dimensão que tendem a residir em variedades matemáticas de baixa dimensão. Este entendimento permite que as redes neurais sejam projetadas para extrair e aprender eficientemente a essência dos dados sem serem sobrecarregadas pela vasta quantidade de informação que cada dado pode representar. A habilidade das redes em aproveitar essas estruturas subjacentes simplifica o processo de aprendizado e melhora significativamente a eficiência dos modelos.

A otimização de redes neurais foi abordada com detalhes, considerando como o landscape de perda e os métodos de otimização (como o gradiente descendente), afetam a eficiência e a eficácia do aprendizado. Discutimos a importância de técnicas de normalização e regularização, como Lasso e Ridge, que ajudam a ajustar os modelos para evitar overfitting e garantir que eles generalizem bem a partir de

conjuntos de treinamento para aplicações reais. Esse aprofundamento nas técnicas de otimização não só melhora o entendimento de como as redes aprendem, mas também equipa praticantes e teóricos com ferramentas para enfrentar desafios de aprendizado mais complexos no campo da inteligência artificial.

EMENDAS

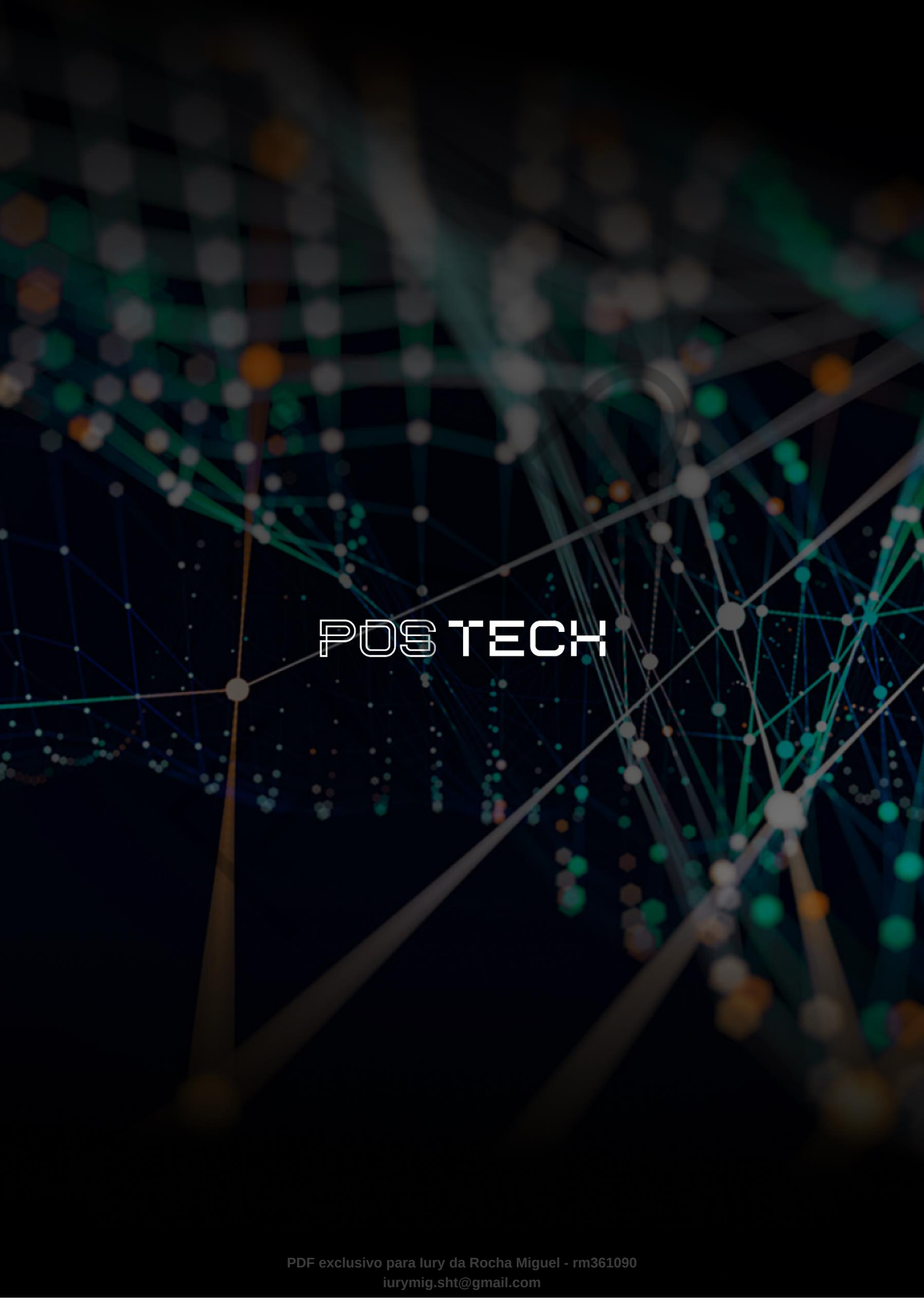
REFERÊNCIAS

- BISHOP, C. M.; BISHOP, H. **Deep Learning**: Foundations and Concepts. [s.l.]: Springer, 2024.
- GROHS, P.; KUTYNIOK, G. **Mathematical Aspects of Deep Learning**. [s.l.]: Cambridge University Press, 2023.
- HUYEN, C. **Designing Machine Learning Systems**: An Iterative Process for Production-Ready Applications. [s.l.]: Springer Science+Business Media, LLC, 2007.
- LIU, Z. et al. **KAN**: Kolmogorov-Arnold Networks. 2024. Disponível em: <<https://arxiv.org/abs/2404.19756>>. Acesso em: 02 ago. 2024.
- MURPHY, K. P. **Machine Learning**: A Probabilistic Perspective. [s.l.]: MIT Press, 2024.
- PRINCE, S. J. D. **Understanding Deep Learning**. [s.l.]: MIT Press, 2024.
- STOLL, E. et al. **Machine Learning and Systems Engineering**. [s.l.]: Springer, 2023.

PALAVRAS-CHAVE

Deep Learning. Redes Neurais. Otimização. Perceptron de Multi-Camadas.

EMEND

The background is a dark, abstract network visualization. It features a complex web of glowing nodes and connecting lines. The nodes are represented by small, semi-transparent spheres in various colors, including teal, orange, and grey. The lines are thin, light-colored threads that crisscross the entire frame, creating a sense of depth and connectivity. The overall effect is reminiscent of a digital or biological network, such as a neural network or a data flow diagram.

POSTECH